Lecture 10

Nomad + Ollama

AI workloads on Nomad — run LLMs privately with open-source tooling

Based on Nomad: AI workloads (HashiCorp Developer)

Where We Are in the Course

Lecture	Topic
8	Terraform — baseline infra (S3, RDS, …)
9	Nomad — `batch` hello world (Docker busybox); optional `exec` on Linux
10	Nomad — schedule **LLM** stacks: **Ollama** (+ UI)

Together: Terraform provisions **nodes** (CPU/RAM/GPU); Nomad **places** Ollama and related jobs on the right clients.

Why Run a Private LLM?

- **Data privacy** — Data stays on **your** infra; not sent to a SaaS for training by default
- **Air-gapped** — Models run **offline** after weights are available; no internet needed per prompt
- **Cost control** — Avoid per-seat SaaS fees; pay for **compute** you control (e.g. EC2 by the hour)

Why Run an LLM on Nomad?

- LLMs are **resource-heavy** — e.g. Ollama suggests ~8GB RAM for ~7B models, more for larger models
- **Cloud** gives elastic RAM/GPU; **Nomad** schedules jobs onto the right **node pool** (CPU vs GPU)
- **Multi-region**, **scalable clients**, **NVIDIA GPU** support
- **Terraform + Nomad** — infra scales with workload; jobspecs stay **declarative**

AI Workload Components (HashiCorp Pattern)

1. **LLM weights** — Packaged model (e.g. **IBM Granite** variants: vision, code, chat)
2. **Ollama** — Loads and serves models; **HTTP API** (`/api/pull` , `/api/generate` , …)
3. **Open WebUI** — Browser UI; talks to Ollama; optional **RAG**, chat history, model switching

Flow: User → **Open WebUI** → **Ollama** → model on disk / memory

Jobs: `ollama` (Conceptual)

- **Type:** `service`
- **Task** `ollama-task`: Docker `ollama/ollama`, port **11434**, **Nomad service** `ollama-backend`
- **Resources:** Tutorial example ~9.1 GHz CPU, ~15 GB RAM (tune to your cluster)
- **Optional:** `node_pool = "large"` — target beefy nodes (Nomad 1.6+ style pools)

Production tutorials often split **node pools** (large for Ollama, small/public for UI).

Jobs: Pull Model (`poststart`)

- **Task** `download-granite3.3-model`: `driver = "exec"`, `lifecycle { hook = "poststart" }`
- After Ollama starts, run `curl` against `/api/pull` with JSON `{"name": "granite3.3:2b"}`
- `nomadService "ollama-backend"` — Injects **address:port** into `template` → **env** for the script

This pattern: **service up first**, then **one-shot side effect** (pull weights).

Jobs: Open WebUI (Conceptual)

- **Service** on nodes with `meta.isPublic = true` (UI on “edge” clients)
- `nomadService "ollama-backend"` — Sets `OLLAMA_BASE_URL`
- **Optional:** `nomadVar` — S3 credentials from **Terraform** → Nomad Variables (Lecture 8 + 10)
- **Health check** — HTTP `/` on the UI port
- **Nomad Actions** — e.g. run SQL to seed admin when signup is disabled

Smaller Models for Class / Laptops

Model (example)	Rough RAM hint
tinyllama / small tags	Lower; good for demos
granite3.3:2b	Tutorial default; still needs several GB
Larger 13B+	16GB+ as per Ollama guidance

Assignment: Default jobspec uses a **small** model; you can switch to **Granite** on a cloud node.

Lecture 9 → Lecture 10

- **L9:** `hello-world` batch (busybox); optional `exec` on Linux; optional **nginx service**
- **L10: Two-task group** (Ollama + **poststart** pull), **Nomad native service discovery**, optional second job (UI)

Same ideas: **jobspec**, **Docker driver**, `template`, `service`.

Official Next Steps (HashiCorp)

1. **Configure a Granite AI workload** — Terraform cluster on AWS, node pools
2. **Run a Granite AI workload** — Submit `ollama.nomad.hcl` + `openwebui.nomad.hcl`
3. **Scale node pools** — Match capacity to inference load

Companion repo (HashiCorp): jobspecs **`ollama.nomad.hcl`**, **`openwebui.nomad.hcl`**.

Summary

- **Private LLM** on Nomad = **privacy**, **air-gap**, **cost** tradeoffs you control
- **Ollama** serves models; **Open WebUI** is the chat front end
- **poststart** + `nomadService` = classic **bootstrap** pattern
- **Terraform** (L8) + **Nomad** (L9–L10) = **infra + AI placement**

Hands-on: See `LECTURE10_ASSIGNMENT_README.md` and `assignment/`.